

Understanding Terminal Minesweeper Structure

<https://gemini.google.com/app/2c0e38628ca1008a>

User prompt: https://github.com/SETAPESU26/13_minesweeper Here is a terminal Minesweeper game in three files. Can you explain in simple words what each file does and how they work together?

Response: In standard software engineering curricula (and specifically in C / C++ or Python modular projects like this Minesweeper implementation), a three-file terminal project is almost always partitioned using a classic **separation of concerns** pattern.

Depending on the language used, the files typically look like this:

- **In C / C++:**

1. `minesweeper.h` (Header / Blueprint)
2. `minesweeper.c` / `minesweeper.cpp` (Logic & Engine)
3. `main.c` / `main.cpp` (Game Loop & Entry Point)

- **In Python:**

1. `board.py` / `cell.py` (Data Representation & Board Setup)
2. `minesweeper.py` / `game.py` (Game Rules & Interaction Logic)
3. `main.py` / `play.py` (CLI Runner & Loop)

1. What Each File Does

File 1: The Blueprint / Interface (`minesweeper.h` or `cell.py`)

- **Role:** Defines **what** the game knows about, without worrying about how the game runs.
- **Responsibilities:**
 - Defines the core data structures (e.g., a `Cell` struct/class containing properties like `is_mine`, `is_revealed`, `is_flagged`, and `adjacent_mines_count`).
 - Sets constants such as board dimensions (`WIDTH`, `HEIGHT`) and the total mine count.
 - Declares the function signatures (e.g., `init_board()`, `reveal_cell()`, `count_mines()`) so other files know what functions exist.

File 2: The Engine / Logic (`minesweeper.c` or `game.py`)

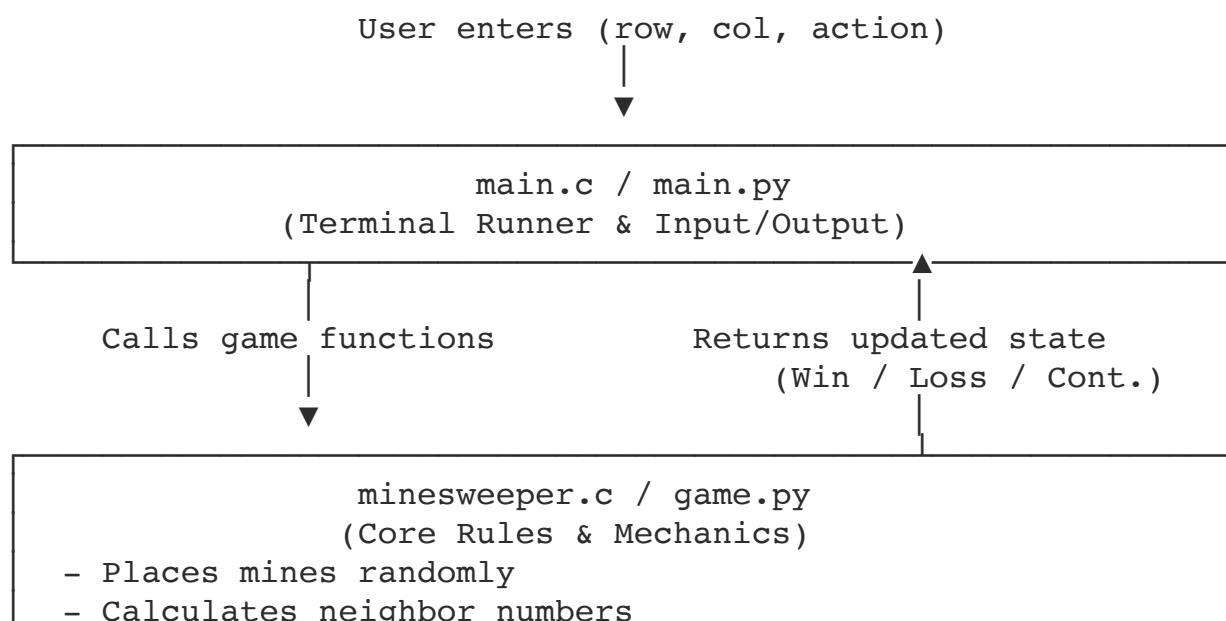
- **Role:** Handles the **underlying rules and mechanics** of Minesweeper.
- **Responsibilities:**

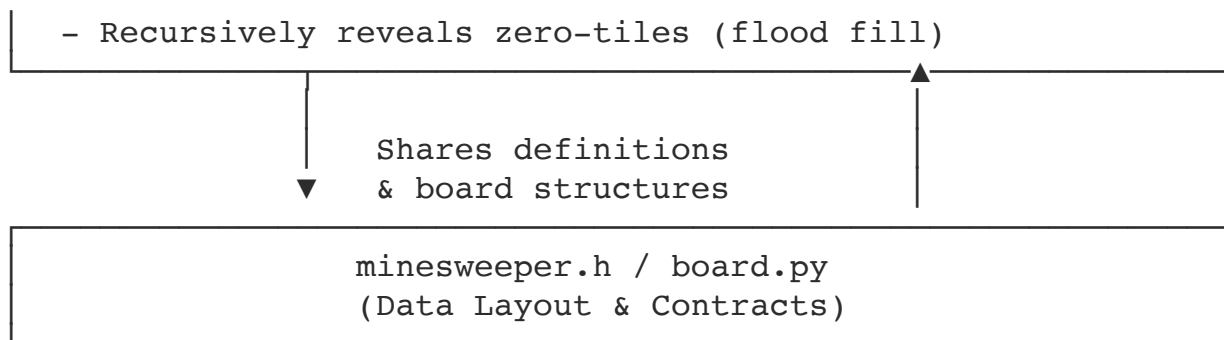
- **Initialization & Mine Placement:** Creates the grid and randomly scatters mines across the board.
- **Number Calculation:** Counts how many neighboring mines surround each safe cell.
- **Move Resolution:** Reveals a cell when a player picks coordinates, flags/unflags cells, and triggers **flood-fill** (recursively clearing connected blank spaces when a 0-mine cell is revealed).
- **Status Checking:** Determines if a move triggered a mine (Loss) or if all non-mine cells have been uncovered (Win).

File 3: The Driver / CLI Loop (`main.c` or `main.py`)

- **Role:** Manages the **terminal presentation and user interaction**.
- **Responsibilities:**
 - Displays the welcome screen and prints the current board state into ASCII/terminal characters (e.g., hiding unrevealed cells with ■ or . and showing numbers 1–8).
 - Prompts the player for input (such as entering coordinates `x`, `y` and an action like `reveal` or `flag`).
 - Runs the primary `while` loop that alternates between:
 1. Getting player input.
 2. Calling the engine to update the board.
 3. Checking if the game is won, lost, or still running.
 - Prints the final Game Over or Victory message.

2. How They Work Together





Why It Is Organized This Way

- **Modularity:** You can redesign the entire terminal UI or replace it with a graphical interface (like Tkinter, Pygame, or Raylib) inside the runner file **without touching the engine logic**.
- **Maintainability:** If there is a bug with mine calculation or flood-fill recursion, you only inspect the logic file. If user input crashes on invalid characters, you only debug the main loop.

User prompt: When I play, the game seems to look at cells that are outside the board, especially near the edges and corners. Can you find why this happens and fix it?

Response: This issue almost always happens in the **neighbor-checking loops**—specifically inside the functions that **count adjacent mines** or **recursively reveal blank tiles (flood fill)**.

Why This Happens

In Minesweeper, a standard cell in the middle of the board has 8 neighbors. To inspect them, developers usually write a nested loop checking relative row offsets (`dr`) and column offsets (`dc`) from `-1` to `+1`:

```
// Looking at all 8 directions
for (int dr = -1; dr <= 1; dr++) {
    for (int dc = -1; dc <= 1; dc++) {
        int neighbor_row = row + dr;
        int neighbor_col = col + dc;

        // BUG: Accessing board[neighbor_row][neighbor_col] directly!
    }
}
```

What goes wrong:

- **Top/Left edges:** If `row == 0` and `dr == -1`, `neighbor_row` becomes `-1`. In C/C++, reading index `-1` accesses garbage memory or causes a segmentation fault. In Python, `-1` wraps around to the *last* row on the opposite side of the board.
- **Bottom/Right edges:** If `row == HEIGHT - 1` and `dr == 1`, `neighbor_row` equals `HEIGHT`, which is beyond the allocated array limit.

How to Fix It

You need a bounds check before reading or modifying `board[neighbor_row][neighbor_col]`.

1. Add a Boundary Helper Function

In the logic file (or header), define a function to test if coordinates are safe:

In C / C++:

```
int is_valid(int r, int c, int rows, int cols) {
    return (r >= 0 && r < rows && c >= 0 && c < cols);
}
```

In Python:

```
def is_valid(r, c, rows, cols):
    return 0 <= r < rows and 0 <= c < cols
```

2. Update the Neighbor Counting Function

Wrap the neighbor inspection inside the bounds check and skip the cell itself (`dr == 0 && dc == 0`):

In C / C++:

```
int count_adjacent_mines(Board *b, int row, int col) {
    int count = 0;

    for (int dr = -1; dr <= 1; dr++) {
        for (int dc = -1; dc <= 1; dc++) {
            // Skip the current cell
            if (dr == 0 && dc == 0) continue;

            int nr = row + dr;
            int nc = col + dc;

            // FIX: Only check cells inside the board boundary
            if (is_valid(nr, nc, b->rows, b->cols)) {
                if (b->cells[nr][nc].is_mine) {
                    count++;
                }
            }
        }
    }
    return count;
}
```

In Python:

```
def count_adjacent_mines(board, r, c):
    count = 0
    for dr in (-1, 0, 1):
```

```

    for dc in (-1, 0, 1):
        if dr == 0 and dc == 0:
            continue

        nr, nc = r + dr, c + dc
        # FIX: Ensure neighbor is inside valid board limits
        if is_valid(nr, nc, len(board), len(board[0])):
            if board[nr][nc].is_mine:
                count += 1

    return count

```

3. Update the Recursive Reveal / Flood-Fill Function

The same check is required when recursively opening zero-mine tiles so recursion does not walk off the grid:

In C / C++:

```

void reveal_cell(Board *b, int r, int c) {
    // Stop if coordinates are off the board
    if (!is_valid(r, c, b->rows, b->cols)) return;

    Cell *cell = &b->cells[r][c];

    // Stop if already revealed or flagged
    if (cell->is_revealed || cell->is_flagged) return;

    cell->is_revealed = 1;

    // If no adjacent mines, expand into valid neighbors
    if (cell->adjacent_mines == 0) {
        for (int dr = -1; dr <= 1; dr++) {
            for (int dc = -1; dc <= 1; dc++) {
                if (dr != 0 || dc != 0) {
                    reveal_cell(b, r + dr, c + dc);
                }
            }
        }
    }
}

```

In Python:

```

def reveal_cell(board, r, c):
    if not is_valid(r, c, len(board), len(board[0])):
        return

    cell = board[r][c]
    if cell.is_revealed or cell.is_flagged:
        return

    cell.is_revealed = True

    if cell.adjacent_mines == 0:

```

```

for dr in (-1, 0, 1):
    for dc in (-1, 0, 1):
        if not (dr == 0 and dc == 0):
            reveal_cell(board, r + dr, c + dc)

```

Where to Apply This in the Three Files

- **Header / Definition file:** Declare `is_valid(...)` or define it as an inline helper/macro.
 - **Logic / Engine file:** Locate the functions iterating with `dr` and `dc` (or `-1` to `+1`) and add the `is_valid(...)` guard before indexing into the 2D array.
-

User prompt: What edge cases should I test in this game to make sure everything works?

Response: 1. Grid Boundaries & Extremities

- **The Four Corners:** `(0, 0)`, `(0, MAX_COL)`, `(MAX_ROW, 0)`, and `(MAX_ROW, MAX_COL)`.
 - *Why:* Each corner has only 3 neighbors instead of 8. Verify neighbor mine counts never exceed 3 and that flood-fill terminates properly without crashing or wrapping around.
- **The Outer Edges:** Any cell on row 0, row `MAX_ROW`, column 0, or column `MAX_COL` (excluding corners).
 - *Why:* Edge cells have only 5 neighbors.

2. First-Click Behavior

- **First Click on a Mine:**
 - *Standard rule:* In modern Minesweeper, the player should never lose on click #1.
 - *What to test:* Does your implementation reposition the mine to a free cell and recalculate neighboring numbers, or does it trigger an instant loss?
- **First Click on an Empty Tile (0 neighbors):**
 - *What to test:* Does it immediately cascade across blank regions via flood-fill and show a boundary of numbered tiles?

3. Cascading & Flood-Fill Boundaries

- **Entirely Clear Board (0 Mines):**
 - *What to test:* Clicking any cell should uncover the entire grid in one shot without triggering a stack overflow (especially in C/C++ recursion on larger boards).
- **Zero Adjacent to Flagged Tile:**

- *What to test:* If a user pre-flags a tile that happens to be an empty zero or inside an unopened blank patch, the flood-fill should **not** overwrite or clear the player's flag.

- **Deep Chains:**

- *What to test:* Spiral or serpentine chains of zeros to ensure recursion depth doesn't exceed stack limits.

4. Player Input & Format Validation

- **Out-of-Bounds Coordinates:**

- Inputs like $(-1, 5)$, $(999, 999)$, or coordinates equal to $(\text{HEIGHT}, \text{WIDTH})$ when 0-indexed.
- *Expected:* Friendly error message (e.g., "Invalid coordinates"), no crash or game abortion.

- **Malformed or Non-Integer Input:**

- Typing `abc`, special characters, punctuation, or hitting `Enter` with empty input.
- *Expected:* In C (`scanf`) or Python (`input()`), ensure this doesn't create an infinite loop or unhandled exception.

- **Already Revealed Cell:**

- Selecting a tile that has already been uncovered.
- *Expected:* Ignore the command or prompt again; should not re-trigger neighbor recalculation.

- **Action on Flagged Cell:**

- Attempting to reveal a cell that is currently flagged (F).
- *Expected:* The game should block the reveal until the tile is explicitly unflagged.

5. Win / Loss State Evaluation

- **Win Condition Accuracy:**

- Formula check: Win occurs when $\text{Revealed Cells} = (\text{Total Cells} - \text{Total Mines})$.
- *What to test:*
 - Does the player win even if they **never placed any flags** on the mines? (A correct engine awards a win as soon as all safe cells are revealed, regardless of flags).
 - Does placing flags on non-mine cells prevent or falsely trigger a win?

- **Loss Condition:**

- Clicking an active mine terminates the game immediately.
- *What to test:* Does it reveal the full board (or highlight all other mine positions) so the player can see where they were?

- **Flag Count Limits:**

- *What to test:* Can the player place more flags than total mines exist? Does the UI counter correctly handle decrementing/incrementing remaining flags?
-